



EMBEDDED SOFTWARE ENGINEERING

Session 3 – Separating BSP from Application Code

Laurent Fiack – laurent.fiack@ensea.fr

Bureau D212

Today

- Last time: the superloop – `main()` as background, interrupts as foreground
- Everything still talks to HAL directly, everywhere – LEDs, buttons, whatever comes next
- Today: pull hardware access apart from application logic, in 3 small steps, on `td2`
- Then: why it's worth it – the same application code running on the board *and* on a desktop simulator
- Last: a quick look at layering *inside* a driver (MCU vs. IC)

Today

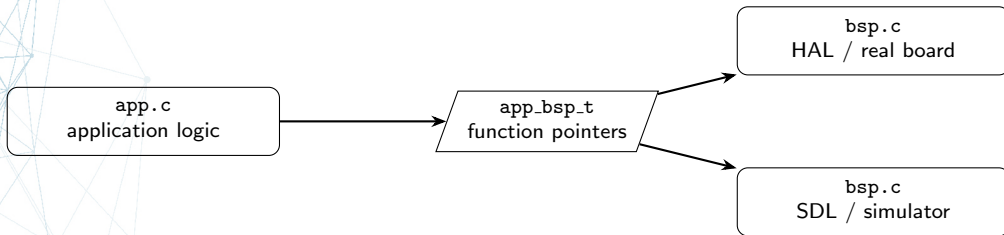
- 1 Why Separate?
- 2 Step 1: A Flat BSP
- 3 Step 2: Application State
- 4 Step 3: The Real Separation
- 5 The Payoff

Where we left off

```
/* td2/Core/Src/main.c -- everything talks to HAL directly */
while (1) {
    uint32_t tick = HAL_GetTick();
    if (0 == (tick % LED_BLINK_PERIOD)) {
        HAL_GPIO_TogglePin(LED1_GPIO_Port, LED1_Pin);
    }
    if (sw1_pressed) {
        sw1_pressed = false;
        HAL_GPIO_TogglePin(LED2_GPIO_Port, LED2_Pin);
    }
    /* ... */
}
```

- Works fine for 3 LEDs and 2 buttons – but every line here is tied to this exact MCU, this exact HAL

The destination



- Same app.c, unmodified, either side – only main.c decides which BSP it's wired to
- That's the whole point of today: get here, in steps you understand, not by magic

BSP = Board Support Package

Set of board-specific functions.

First move: name the hardware

- Pull every HAL_* call out of `main.c` into functions with a job-specific name
- No structure yet – just functions. Private state (button edge flags) stays `static` inside the file
- That `static` is exactly the encapsulation from session 1 – module-private state, only reachable through the functions we choose to expose

Setting up the bsp/ folder

- New source folder at the project root, next to Core/: right-click the project → New → Source Folder → name it bsp
- Inside it, two new files: bsp.h and bsp.c (right-click bsp/ → New → Header/Source File)
- CMakeLists.txt lives at the project root too – it needs to be told about the new folder before anything in it gets compiled

Live demo

We create the folder and both files together, right now, on td2 – the content comes next.

Wiring it into the build

```
target_sources(${CMAKE_PROJECT_NAME} PRIVATE
    # Add user sources here
+   bsp/bsp.c
)

target_include_directories(${CMAKE_PROJECT_NAME} PRIVATE
    # Add user defined include paths
+   bsp/
)
```

- Two edits, both in CMakeLists.txt: add bsp.c to the sources, add bsp/ to the include paths – otherwise #include "bsp.h" won't resolve anywhere else in the project
- Re-run CMake configure (CubeIDE does this automatically on save) before the next build

bsp.h

```
void bsp_init(void);  
void bsp_process(void);  
  
void bsp_led_set(uint8_t led, bool on);  
bool bsp_sw1_pressed(void);  
bool bsp_sw2_pressed(void);  
uint32_t bsp_get_tick_ms(void);
```

- Named after what they do, not after the register they touch

bsp.c

```
static volatile bool sw1_pressed = false;
static volatile bool sw2_pressed = false;

void HAL_GPIO_EXTI_Callback(uint16_t pin) {
    if (pin == BTN_SW1_Pin) { sw1_pressed = true; }
    if (pin == BTN_SW2_Pin) { sw2_pressed = true; }
}

void bsp_led_set(uint8_t led, bool on) {
    GPIO_PinState state = on ? GPIO_PIN_SET : GPIO_PIN_RESET;
    switch (led) {
        case 0: HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, state); break;
        /* ... */
    }
}

bool bsp_sw1_pressed(void) {
    bool edge = sw1_pressed;
    sw1_pressed = false;
    return edge;
}

/* ... */
```

main.c at this stage

```
bsp_init();  
while (1) {  
    bsp_process();  
    /* the "what to do about it" still lives right here, for now */  
    if (bsp_sw1_pressed()) {  
        bsp_led_set(1, true);  
    }  
}
```

- Same superloop as session 2 – just talking to named hardware instead of raw pins
- Nothing about *when* to blink or *what a button press means* has moved yet

Second move: give the app its own state

- Is LED1 on right now? Since when? – that's *application* state, not hardware state
- It deserves its own structure: `app_t`
- Not decoupled yet: `app.c` still `#include "bsp.h"` directly

Same recipe: an app/ folder

- Same three moves as bsp/ a moment ago: new source folder app/, then app.h/app.c inside it

```
target_sources(${CMAKE_PROJECT_NAME} PRIVATE
    bsp/bsp.c
+   app/app.c
)
```

```
target_include_directories(${CMAKE_PROJECT_NAME} PRIVATE
    bsp/
+   app/
)
```

app.h / app.c

```

/* app.h */
#include "bsp.h"    /* direct coupling, for now */

typedef struct {
    uint32_t last_tick;
    bool led1_on, led2_on, led3_on;
} app_t;

void app_init(app_t *app);
void app_process(app_t *app);

/* app.c */
void app_process(app_t *app) {
    uint32_t tick = bsp_get_tick_ms();
    if (tick - app->last_tick >= 250) {
        app->last_tick = tick;
        app->led1_on = !app->led1_on;
        bsp_led_set(0, app->led1_on);
    }
    /* led2_on/led3_on on bsp_sw1_pressed()/bsp_sw2_pressed(): same shape */
}

```

main.c becomes trivial

```
app_t app;  
bsp_init();  
app_init(&app);  
while (1) {  
    bsp_process();  
    app_process(&app);  
}
```

- All the "what to do" logic now lives in `app_process` – readable on its own

One question before we go further

Can this `app.c` run on anything other than the board – a simulator, say?

- No: it `#includes bsp.h`, which pulls in HAL
- Compiling `app.c` natively would drag `bsp.c` and all of HAL along with it
- That's the problem step 3 solves

bsp_t: the interface, not the implementation

```
typedef struct bsp_struct bsp_t;  
  
struct bsp_struct {  
    void * ctx;  
    void (*led_set)(bsp_t * bsp, uint8_t led, bool on);  
    bool (*sw1_pressed)(bsp_t * bsp);  
    bool (*sw2_pressed)(bsp_t * bsp);  
    uint32_t (*get_tick_ms)(bsp_t * bsp);  
};
```

- A struct of function pointers plus an opaque ctx – app.c includes *this*, never bsp.h
- It never needs to know what ctx really points to

bsp.c: the actual implementation

- bsp.c gets an instance structure, bsp_ctx_t – the state that used to be static moves inside it
- Its functions now take a ctx through bsp_t

```
typedef struct bsp_ctx_struct {  
    volatile bool sw1_pressed;  
    volatile bool sw2_pressed;  
} bsp_ctx_t;  
  
static void bsp_led_set(bsp_t * bsp, uint8_t led, bool on);  
static bool bsp_sw1_pressed(bsp_t * bsp);  
static bool bsp_sw2_pressed(bsp_t * bsp);  
static uint32_t bsp_get_tick_ms(bsp_t * bsp);
```

bsp_init links the interface to the implementation

```
void bsp_init(bsp_t * bsp) {  
    bsp->ctx = (void*)&bsp_ctx;  
  
    bsp_ctx.sw1_pressed = false;  
    bsp_ctx.sw2_pressed = false;  
  
    bsp->led_set = bsp_led_set;  
    bsp->sw1_pressed = bsp_sw1_pressed;  
    bsp->sw2_pressed = bsp_sw2_pressed;  
    bsp->get_tick_ms = bsp_get_tick_ms;  
}
```

app.c: the logic barely changes

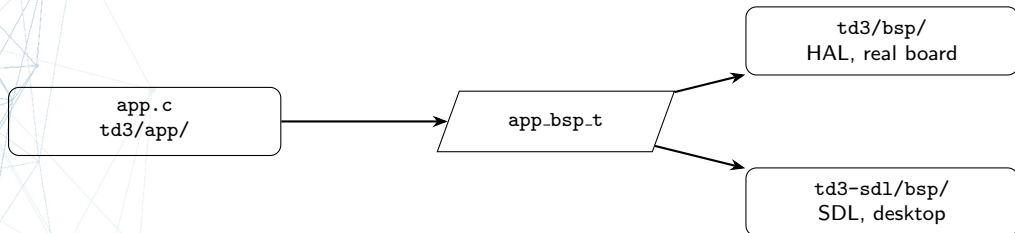
```
/* step 2 */  
bsp_led_set(1, app->led2_on);  
  
/* step 3 */  
bsp_t * bsp = app->bsp;  
bsp->led_set(bsp, 1, app->led2_on);
```

- Same decisions, same order, same variables – only *how* we reach the hardware changed
- That's reassuring to say out loud: separating BSP from app is mostly mechanical once the shape is right

main.c: the only place that still knows

```
int main (void) {  
    app_t app;  
    bsp_t bsp;  
  
    bsp_init(&bsp);  
    app_init(&app, &bsp);  
  
    while (1) {  
        bsp_process(&bsp);  
        app_process(&app);  
    }  
  
    return 0;  
}
```

Same app.c, two BSPs



- `td3-sd1/` doesn't copy `app.c` – its build references `td3/app/` directly, by relative path. One file, two targets, no fork to keep in sync.

Live demo

The board and the simulator, side by side, running the exact same `app.c`. We won't look at the SDL code in detail – the point is that it's possible, not how SDL works.

Take away

- Separation isn't a leap – it's 3 small, working steps: name the hardware, give the app a home, then swap direct calls for an interface
- The payoff is concrete: one `app.c`, board or simulator, `main.c` picks
- Next session: now that behaviour lives in `app.c` as plain functions, we look at structuring it with encapsulation, inheritance and polymorphism – OOP in C